

Hierarchical Deep Reinforcement Learning for VWAP Strategy Optimization

Xiaodong Li , Pangjing Wu , Chenxin Zou , and Qing Li , *Fellow, IEEE*

Abstract—Designing algorithmic trading strategies targeting volume-weighted average price (VWAP) for long-duration orders is a critical concern for brokers. Traditional rule-based strategies are explicitly predetermined, lacking effective adaptability to achieve lower transaction costs in dynamic markets. Numerous studies have attempted to minimize transaction costs through reinforcement learning. However, the improvement for long-duration order trading strategies, such as VWAP strategy, remains limited due to intraday liquidity pattern changes and sparse reward signals. To address this issue, we propose a jointed model called Macro-Meta-Micro Trader, which combines deep learning and hierarchical reinforcement learning. This model aims to optimize parent order allocation and child order execution in the VWAP strategy, thereby reducing transaction costs for long-duration orders. It effectively captures market patterns and executes orders across different temporal scales. Our experiments on stocks listed on the Shanghai Stock Exchange demonstrated that our approach outperforms optimal baselines in terms of VWAP slippage by saving up to 2.22 base points, verifying that further splitting tranches into several subgoals can effectively reduce transaction costs.

Index Terms—Algorithmic trading, deep learning, hierarchical reinforcement learning, optimized trade execution.

I. INTRODUCTION

WHEN executing a large order (*a.k.a.* a parent order), brokers may encounter a large slippage due to market impacts. The sudden execution of the large order causes market prices to deviate from expected levels, resulting in higher transaction costs. Algorithmic trading aims to optimize order execution by employing computer-programmed rules. It involves splitting the parent order into multiple smaller orders (*a.k.a.* child orders) and executing them based on predetermined rules over time.

In 1988, Berkowitz et al. [1] suggested using the difference between market volume-weighted average price (VWAP) and orders' VWAP as a metric for the transaction costs, *a.k.a.*

Manuscript received 4 March 2023; revised 20 September 2023; accepted 21 November 2023. Date of publication 30 November 2023; date of current version 15 May 2024. This work was supported in part by the National Natural Science Foundation of China under Grant 61602149, in part by the Central University Basic Research Fund of China under Grant B210202078, and in part by the Postgraduate Research & Practice Innovation Program of Jiangsu Province under Grant KYCX21_0483. Recommended for acceptance by G. Huang. (Corresponding author: Xiaodong Li.)

Xiaodong Li, Pangjing Wu, and Chenxin Zou are with the College of Computer and Information, Hohai University, Nanjing 211100, China (e-mail: xiaodong.li@hhu.edu.cn; pangjing.wu@outlook.com; zoucx@hhu.edu.cn).

Qing Li is with the Department of Computing, Hong Kong Polytechnic University, Hong Kong 999077, China (e-mail: csqli@comp.polyu.edu.hk).

Digital Object Identifier 10.1109/TBDATA.2023.3338011

TABLE I
EXISTING STUDIES OF RL IN ALGORITHMIC TRADING

Author	Execution Time	Total Order Size
Nevmyvaka et al. [11]	2~8 min	5K; 10K
Hendricks and Wilcox [28]	20 ~ 60 min	10K; 1M
Shen et al. [31]	10 min	20K
Ning et al. [12]	60 min	2K
Ye et al. [29]	2 min	5K; 10K
Lin and Beling [4]	2 min	300 ~ 7K
Fang et al. [32]	30 min	< 1K
Wang et al. [33]	20 min	10K~ 40K
Ours	240 min	100K ~ 160K*

* The total order size of our model is proportional to the given minimum trading unit. Increasing the minimum unit allows our model to be adapted to larger orders.

VWAP slippage. Fundamental VWAP strategy was to split the parent order according to uniformly estimated market liquidity, targeting the goal of reducing VWAP slippage [2], [3]. Due to it requiring few assumptions about market microstructures [4], brokers have widely adopted this approach for efficient order execution. However, this strategy was explicitly predetermined and lacked adaptability to dynamic market conditions [5], thus limiting its ability to achieve lower transaction costs. Therefore, researchers began to improve the VWAP strategy from two aspects. One was to improve volume profile estimation [6], [7]. The other was to adjust the trading strategy according to recent historical price and volume information [3], [8], [9], [10]. Nevertheless, these approaches relied on strict modeling and assumptions about the market's microstructures, and the unpredictable market limited their efficiency [4].

Since stock trading involves decision-making processes consistent with the problem addressed by reinforcement learning (RL), many researchers have turned to RL models for optimizing algorithmic trading strategies. Nevmyvaka et al. [11] first demonstrated the efficiency of RL in minimizing transaction costs by a large-scale empirical application, and subsequent studies have leveraged RL models to optimize algorithmic trading strategies [4], [12], [13], [14], [15], [16], [17]. However, most of these studies have focused on short-term strategies executed over a few minutes, as summarized in Table I, neglecting more challenging strategies for long-duration orders, such as the VWAP strategy. As previous research has demonstrated significant daily periodicity in trading intensity [18], [19], fluctuating liquidity patterns throughout the day and even within seconds on limit order books (LOBs) pose challenges for optimal decision-making. Therefore, designing an RL-based trading strategy capable of capturing price and liquidity variations across different time scales is crucial.

Despite recent representative research [20] demonstrating the superior performance of deep Q-learning network (DQN) over human players in most Atari games, it encountered challenges in learning an effective policy for *Montezuma's Revenge*, characterized by diverse goals and sparse rewards. In algorithmic trading, when applying general RL models to optimize trading strategies considering VWAP slippage, we observed instances where the agent exhibited a myopic behavior due to sparse rewards. This resulted in excessive order execution within a short time, ultimately leading to increased transaction costs. A practical approach to this problem is hierarchical reinforcement learning (HRL) [21]. HRL models establish temporal abstractions for the environment and decompose decision processes into simpler subgoals, showcasing adaptability and efficiency in tackling complex RL tasks such as the NP-hard dynamic pickup and delivery problem [22].

By incorporating the idea of hierarchical decision-making in RL-based algorithmic trading strategies, we propose a joint model termed Macro-Meta-Micro Trader (M3T) to optimize parent order allocation and child order execution of the VWAP strategy, thereby reducing the transaction costs of long-duration orders. The M3T captures liquidity patterns and executes orders from different temporal scales through three hierarchical traders: Macro Trader, Meta Trader, and Micro Trader. First, the Macro Trader adopts a long short-term memory (LSTM) network [23] to improve the estimation of volume profiles and allocates a parent order to tranches accordingly. Second, the Meta Trader selects a subgoal appropriate to instant liquidity to form a mini-tranche. Finally, the Micro Trader extracts the market state by a multi-headed self-attention encoder and executes orders to fulfil the subgoal and reduce transaction costs. The optimal trading strategy is achieved through hierarchically optimizing each trader. To verify the efficiency of our approach, we performed extensive experiments over ten stocks listed on the Shanghai Stock Exchange (SSE). The experimental results demonstrated that our M3T outperforms baselines in terms of VWAP slippage on all stocks, with up to 2.22 base points compared to optimal baselines, i.e., double DQN (DDQN) [24]. It verified that further splitting each tranche into several subgoals can reduce transaction costs. Additional analysis also demonstrated that each trader in M3T can effectively reduce transaction costs.

Our main contributions are summarized as follows:

- 1) We propose a joint model called M3T, which combines DL and HRL to optimize the foundational VWAP strategy through three hierarchical traders handling different temporal scales. To our best knowledge, this study is the first to demonstrate the effectiveness of HRL in optimizing the execution of long-duration orders.
- 2) We extend the traditional Markov decision process (MDP) to a hierarchical Markov decision process (HMDP) in algorithmic trading tasks, introducing hierarchical control into the child order execution process.
- 3) Since the microstructure of the Chinese stock market differs significantly from previous studies, we provide a detailed discussion on the effectiveness of M3T. Experimental results based on various DL and RL models further

demonstrate the efficiency of M3T for long-duration order execution.

The rest of this paper is organized as follows. Section II discusses the related studies on algorithmic trading and HRL. Section III introduces the foundational VWAP strategy and proposes HMDP for M3T optimization. Section IV proposes our M3T model. Section V describes the experimental setup and reports the experimental results and related discussions. Section VI provides our conclusions.

II. RELATED WORK

A. Algorithmic Trading

Bertsimas and Lo [25] were the first to use dynamic programming to find the explicit closed-form solution of order execution. Almgren and Chriss [26] proposed the Almgren-Chriss model, extending Bertsimas and Lo's model by adding cost evaluation, price impact functions, and risk aversion parameters. Huberman and Stanzl [27] considered traders' appetite for liquidity risk in order execution. However, these studies make strong assumptions about underlying price movements or distributions, which are difficult to apply to real stock markets. Berkowitz et al. [1] suggested using VWAP slippage as the metric of the transaction cost. They proposed the VWAP strategy that split a parent order based on the market liquidity to reduce the VWAP slippage. However, these static strategies could not obtain optimal costs in a dynamic market. There were two main approaches to enhancing the VWAP strategy. The one was to improve the forecasting of volume profiles. Konishi [6] modeled the fluctuation pattern of price and volume as Brownian movement. Bialkowski et al. [7] proposed a volume forecasting model combining market features and stock features. The other was to improve the strategy using recent price and volume information. Kakade et al. [8] introduced the competitive ratio into the VWAP strategy to reflect the market's liquidity and dynamically adjusted child orders. Almgren and Lorenz [9] minimized the transaction costs via Bayesian estimators of intraday price shifts. However, these approaches mainly relied on the assumptions of markets' microstructures, such as market dynamics, order types, trading rules, etc. Markets' unpredictable patterns limited their capabilities.

As reinforcement learning brought a new way to find the optimal algorithmic trading strategy by automatically learning the underlying dynamic of markets from massive data, many studies began to leverage RL models to optimize trading strategies dynamically. Table I summarizes the existing studies on RL-based algorithmic trading strategies for sale side. The current studies mainly focus on orders with short execution times and small order sizes, which still has a big gap with actual trading task requirements. Nevmyvaka et al. [11] proposed the first large-scale empirical application of optimizing algorithmic trading via RL. They minimized the implementation shortfall (IS) for a buy-side (or sell-side) within a short discrete period. Hendricks and Wilcox [28] optimized IS by using Q-learning to revise the volume derived from the Almgren-Chriss model. Recently, the advent of the DQN [20] has provided a powerful representation ability for RL agents. Ning et al. [12] used DDQN

directly to train trading agents based on high-dimensional LOB data. Ye et al. [29] used convolution neural networks to extract spatial information of LOBs and minimized the transaction costs by deep deterministic policy gradient. Lin and Beling [4] leveraged LSTM networks to extract features from high-dimensional market information as agents' observations and optimized the IS with the proximal policy optimization model. Ye et al. [30] adopted multi-agent RL to optimize trading strategies. The order price and size were decided by different agents respectively. Nevertheless, the existing studies mainly focused on optimizing short-duration order execution and paid less attention to others.

Besides, some studies on portfolio management have also noticed the transaction costs caused by the adjustment of portfolio position in recent years. Fang et al. [32] suggested optimizing algorithmic trading strategies based on knowledge distillation. They used a teacher network with future information to guide a student network with historical data to learn the optimal strategy. Wang et al. [33] proposed a two-layer hierarchical RL model, separating the portfolio management and order execution. The upper agent determined the portfolio position ratio, while the lower agent adjusted the position and minimized transaction costs. However, the position adjustment window was short. These models' effectiveness was only empirically verified on a short time scale. Therefore, designing an RL-based trader sensitive to both micro and macro changes in price and liquidity throughout the day was significant.

B. Hierarchical Reinforcement Learning

Sparse rewards and long execution times presented tremendous challenges for training RL agents. An effective way to address the challenges was by establishing temporal abstractions to decompose decision processes into simpler subgoals. Dayan and Hinton [34] first proposed the idea of multi-level control by dividing the task into three levels: super-manager, manager, and sub-manager. All the levels obeyed two principles: 1) reward hiding, the agent only needed to fulfil the task given by the immediate superior agent rather than considering the completion of the overall task; 2) information hiding, the sub-manager did not need to know the goal issued by the super-manager, and the super-manager did not need to know how the sub-manager achieved the goal. This work provided a prototype for subsequent studies, guiding the design of HRL structures.

After that, Sutton et al. [35] combined the idea of temporal abstraction over action set with MDP and SMDP. In their option framework, the upper agent selects an original action or a combination of original actions as an option in each step, while the lower agent defines a policy over actions for each option. Sorg and Singh [36] optimized the options by combining existing options. Szepesvari et al. [37] optimized the options by using different reward functions. Bacon et al. [38] combined the option framework with the actor-critic structure and directly adopted the policy gradient algorithm to synchronously optimize multi-level control. Besides, some studies have used mutual information rewards to drive low-level agents to actively explore the dynamics of actions and environments without environmental rewards [39], [40], [41].

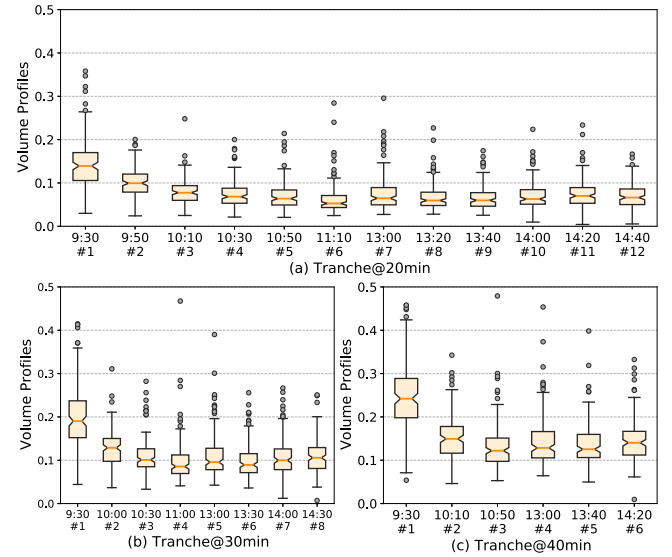


Fig. 1. Historical volume profile distribution from June 2014 to March 2015 of Shanghai Pudong Development Bank (600000). Each box indicates the lower and upper quartiles of trading volume within each tranche. The solid lines illustrate the maximum (minimum) trading volume within the upper (lower) bound, and the dots are the outliers.

From the perspective of state and goal abstractions, Schaul et al. [42] proposed a universal value function for multi-goal optimization problems, which considered goals along with states. Kulkarni et al. [21] proposed a temporal abstract HRL, which divided the overall goal into several subgoals and learned an effective global strategy over each subgoal. It showed good adaptability and efficiency in addressing complex RL tasks. Nachum et al. [43] solved the non-stationary problem between the upper and lower agents in off-policy HRL optimization. Besides, some HRL studies focused on how to learn effective policy based on the idea of hindsight in an environment with sparse rewards. Andrychowicz et al. [44] proposed the hindsight experience replay to select efficient goals under sparse rewards. The effective policy was learned on the failed transition by replacing the unreachable goal with the last reached state. The transition would then receive a positive reward. Since the shift of lower-level policy would affect the dynamic of the upper level, non-stationary problems occurred that previously achievable goals could not be achieved. Levy et al. [45] assumed that the policy at the lower level was already optimal, so changes in the policy at the lower level would not affect the policy at the upper level. Tang et al. [45] extended the HRL to more complex decision problems and discussed the temporal abstraction in multi-agent RL scenarios.

III. PROBLEM FORMULATION

A. Preliminary

Many studies [18], [19] have demonstrated a significant daily periodicity in trading intensity (as shown in Fig. 1), which may affect transaction costs. When the market lacks liquidity, issuing a large order can lead to price deviation from the expected price

or even failure of order execution, thus increasing transaction costs. Algorithmic trading is a class of approaches to reduce transaction costs. It minimizes orders' impacts on the market by allocating large orders into several small tranches according to pre-programmed rules.

The foundational VWAP strategy is a typical impact-driven algorithmic trading strategy focusing on predicting intra-day volume distribution instead of directly modeling the transaction costs [2]. It is suitable for trading orders with large sizes and long execution time. In practice, it allocates a large order into tranches according to the market's volume profiles. Each tranche's volume is consistent with the market liquidity. The expected average trading price is the VWAP,

$$\text{VWAP} = \frac{\sum_{t=1}^T p_t \cdot v_t}{T}, \quad (1)$$

where v_t is the size of order issued at time t , and T is the number of total time steps. The transaction cost is defined by the relative difference between the parent order's VWAP O and the market's VWAP M , known as VWAP slippage, i.e.,

$$\text{VWAP slippage} = \text{side} \cdot \frac{O - M}{M}, \quad (2)$$

where side denotes the buying/selling directions, i.e., $+1$ for the buying and -1 for the selling. A smaller VWAP slippage indicates a lower transaction cost. Under this definition, the objective of order execution is to minimize the VWAP slippage by choosing the best order prices $\{p_t\}_{t=1}^T$ and sizes $\{v_t\}_{t=1}^T$,

$$\min_{\substack{\{(p_t, v_t)\}_{t=1}^T \\ \forall t: p_t \in \mathcal{P}, v_t \in \mathcal{V}}} \frac{\frac{1}{T} \sum_{t=1}^T (p_t \cdot v_t) - M}{M}, \quad (3)$$

where, $\mathcal{P}$ is order price space and $\mathcal{V}$ is order size space.

B. Parent Order Allocation

Before executing a parent order, the VWAP strategy allocates the parent order into several tranches according to the volume profiles, a market liquidity metric represented by the proportion of each tranche's trading volume to the day's volume. The granularity is an adjustable parameter in depicting the volume profile. Fig. 1 shows the volume profiles under three different granularities. It can affect the accuracy of liquidity estimation, as fine-grained volume profiles contain more outliers and noise. In contrast, coarse-grained volume profiles eliminate daily periodicity and present a uniform distribution. Considering that the trading time of the Chinese stock market is 4 hours and the fine-grained volume profiles may contain noise, we divide a full-day execution order $\mathcal{T}$ into eight 30-minute tranches as Fig. 1(b). Specifically, on the day t , the VWAP strategy divides the last n days' trading volume into eight 30-minute tranches. It estimates the volume profiles $\widehat{vp}_t^{(i)}$ of tranche $\#i$ based on its past n days' volume profiles $X_t^{(i)} \doteq \langle vp_{t-n}^{(i)}, vp_{t-n+1}^{(i)}, \dots, vp_{t-1}^{(i)} \rangle$, where the volume profiles are estimated by moving average (MA) method in traditional VWAP strategy. According to the estimated volume profiles, the parent order is allocated into tranches. The order size of each tranche is $\mathcal{T}_t^{(i)} = \mathcal{T} \times \widehat{vp}_t^{(i)}$.

C. Trading Simulation

Based on the microstructure of the Chinese stock market, we build a trading simulation system for order execution. In this system, an RL agent can only issue one child order to align actions with corresponding rewards at each trading step. The old unfulfilled order will be automatically canceled when a new order is issued. The new order will enter different order queues according to its price. Take a sell order as an example:

- If its price is on the buy-side, the order goes to the front of the queue and trades with buyers directly.
- If its price is on the sell-side, the order waits at the end of the queue. The position of the order is updated one step forward when recording a trade whose price is greater than or equal to the order.

However, we cannot accurately estimate orders' impact on the market. We assume that the environment dynamics are consistent with the historical trading data as existing studies did [4], [11], [12], [29]. That is, the agent's decisions do not affect the state of the next time step (but affect the feedback reward). This is because we do not have the knowledge to estimate the impact of a non-existent order on historical situations correctly. In addition, it is also impossible to correctly estimate the length of the order waiting queue based on the historical LOB and trade data because we do not know how many orders are canceled or replaced between each snapshot. Simply ignoring the queue length may make the agent too optimistic about the market liquidity [46]. Therefore, we introduce an empirical constant ι to estimate the queue length. When the order's price crosses the spread, it needs to wait for the previous ι orders to be traded before matching. Its position will move one step forward when an order with the same price is completed in the trade data. Once the order reaches the head of the queue, it is matched with all the quotes whose price is greater than or equal to the order price.

D. Hierarchical Markov Decision Process

MDP provides a mathematical model for training RL agents. In this paper, we extend traditional MDP to HMDP in order to facilitate training M3T. The HMDP consists of extrinsic state $\tilde{\mathcal{S}}$, intrinsic state $\mathcal{S}$, subgoal $\mathcal{G}$, action $\mathcal{A}$, extrinsic reward $\tilde{\mathcal{R}}$, intrinsic reward $\mathcal{R}$, and the state transition probability $\mathcal{F}$. Each element is defined as follows.

Extrinsic State: $\tilde{\mathcal{S}}$ is the state observed by the Meta Trader. It consists of the market's liquidity, tranche execution progress, and overall market trading volume during the last subgoal execution.

Intrinsic State: $\mathcal{S}$ is the state observed by the Micro Trader. It consists of LOBs, subgoal g , and subgoal execution progress, where the LOBs consist of the last 20 LOB snapshots (which are padded with 0 if the number of snapshots is less than 20); the subgoal g is the one-hot embedding of the subgoal given by Meta Trader; the subgoal execution progress consists of time progress and filling ratio of g .

Subgoal: The action $g = (T_g, V_g)$ of the Meta Trader is named *subgoal*. It gives the maximum execution step T_g and order size V_g that the Micro Trader should execute. The subgoal space $\mathcal{G}$ is spanned based on the uniform execution speed. Considering

a 30-minute tranche containing 600 discrete time steps, it must fulfil about 16% of the orders for every 100 steps according to the uniform execution speed. We fluctuate the steps and order sizes by 20% and obtain nine subgoals adapted to the different market liquidity. These subgoals are shown in (4) and numbered from #1 to #9 line by line.

$$\mathcal{G} = \begin{bmatrix} 1\#(80, 0.13) & 2\#(80, 0.16) & 3\#(80, 0.19) \\ 4\#(100, 0.13) & 5\#(100, 0.16) & 6\#(100, 0.19) \\ 7\#(120, 0.13) & 8\#(120, 0.16) & 9\#(120, 0.19) \end{bmatrix}. \quad (4)$$

Action: The action space is spanned by limit orders represented as $\mathcal{A} = \{(p, v) | p \in \mathcal{P}, v \in \mathcal{V}\}$. In each step, the agent can issue a limit order to replace the old order or wait for the old order to be fully executed (i.e., *do nothing*). The limit order's price space $\mathcal{P}$ consists of the current *bid1* or *ask1* price on the LOB, allowing the order to cross the spread for a better trading price. The order's size space $\mathcal{V}$ is fixed to one lot (i.e., 100 shares), which is the minimum trading unit in the Chinese stock market.

Extrinsic Reward: The objective of RL agents is to maximize cumulative rewards. According to Sutton et al. [47], the reward should directly reflect the goal rather than how to achieve the goal. Therefore, we directly use the negative VWAP slippage of the particular subgoal (for M3T) or tranche (for others) as the extrinsic reward $\tilde{r} \in \mathcal{R}$ because our goal is to minimize the VWAP slippage. The Meta Trader will receive a VWAP slippage when the subgoal is fulfilled and 0 otherwise, that is,

$$\tilde{r} = \begin{cases} \text{VWAP slippage (equation 2),} & \text{Fulfil subgoal;} \\ 0, & \text{Otherwise.} \end{cases} \quad (5)$$

Intrinsic Reward: The definition of intrinsic reward $r \in \mathcal{R}$ is similar to (5). It gives 0 at each step and returns the VWAP slippage of the particular subgoal (for M3T) or tranche (for others) when the subgoal is terminated. In addition, we ignore the commission fees in rewards because the commission fees in the Chinese stock market are fixed and do not affect the evaluation of transaction costs.

Penalty: In reality, brokers must execute all the orders within a given time. Otherwise, it will result in a severe consequence of default. Agents can learn this requirement by a penalty reward significantly smaller than general rewards [5]. Suppose there is still inventory at the end of each subgoal. In that case, the Meta Trader or the Micro Trader will receive a penalty of -99, significantly smaller than general rewards within the range of ± 10 . The environment will impose a corresponding penalty based on order size and market liquidity. Specifically, for a given subgoal g , the Meta Trader is considered to give an inappropriate subgoal if the minimum step of fulfilling the subgoal (equal to the order size V_g) exceeds 30% of the maximum step T_g or the order size V_g exceeds 5% of the market volumes v^M . Otherwise, the Micro Trader will be punished, that is,

$$\text{Penalty} = \begin{cases} \tilde{r} = -99, & V_g > 0.3 \cdot T_g; \\ \tilde{r} = -99, & V_g > 0.05 \cdot v^M; \\ r = -99, & \text{Otherwise.} \end{cases} \quad (6)$$

State Transition: Since our environment is a simulation of the real Chinese stock market based on historical data, we cannot

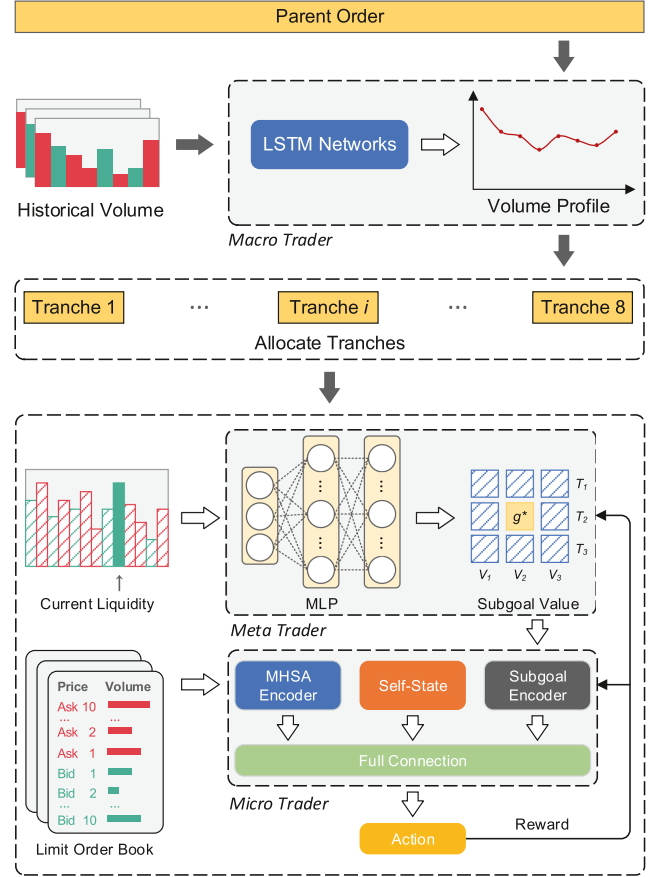


Fig. 2. Scheme of the M3T for VWAP strategy optimization. A parent order is first allocated into tranches based on volume profiles predicted by Macro Trader. Within each tranche, Meta Trader then selects a subgoal appropriate to the current liquidity for Micro Trader, which executes orders based on raw LOB data to fulfil the subgoal. Finally, Meta Trader and Micro Trader optimize their policies according to the feedback rewards.

correctly estimate the impact of our agent's decision on the market. However, a liquid market will rebound to balance after impacting [4], because the child order issued by our agents is tiny. Therefore, we assume that $\mathcal{F}$ is consistent with the history as we discussed in Section III-C. In addition, when the Micro Trader finally fails to fulfil the subgoal, the remainder is liquidated as a market order.

IV. MACRO-META-MICRO TRADER

In this section, we propose our M3T model, which consists of Macro Trader, Meta Trader, and Micro Trader, as illustrated in Fig. 2.

A. Macro Trader

To reduce parent orders' impact on the market, we leverage Macro Trader to forecast volume profiles and allocate parent orders into tranches. It leverages neural networks' powerful representation learning ability to improve the accuracy of volume profile forecast. Since the trial results in Table V show that LSTM [23] performs the best in volume profile forecast, we use

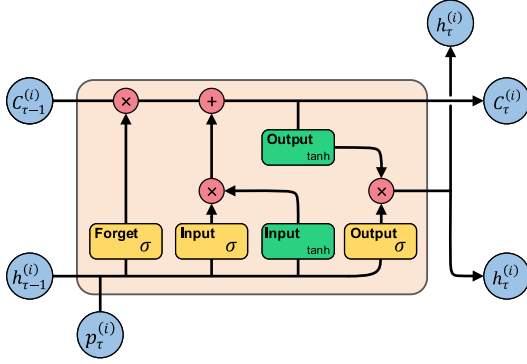


Fig. 3. Example of using LSTM to forecast the i th tranche's volume profile $p_t^{(i)}$ in day t based on last n days' volume profiles. At time step $\tau \in [t - n, t - 1]$, the inputs of the LSTM unit are current volume profile $vp_\tau^{(i)}$, last hidden state $h_{\tau-1}^{(i)}$ and cell state $C_{\tau-1}^{(i)}$. The LSTM first decides to discard which information from the cell state by the forget gate. Then, the input gate obtains temporal information from inputs to form a candidate cell state. Finally, the output gate decides to output which parts of the cell state as the temporal representation of volume profiles.

it as Macro Trader's backbone model. The LSTM network is a recurrent neural network (RNN) widely used to capture temporal patterns. Unlike traditional RNN [48] that only contains recurrent information flow, an LSTM unit consists of several cells (the yellow blocks in Fig. 3), i.e., a forget gate, an input gate, and an output gate, to control the input and output of temporal information. These gate structures overcome the vanishing and explosion of gradient encountered by the RNN in extracting temporal representations.

The Macro Trader first uses the LSTM network to extract the temporal pattern of the last n days' historical volume profiles tranche-by-tranche, as illustrated in Fig. 3. Then a full connection (FC) layer maps the hidden state output by the LSTM to estimate the volume profiles $\langle vp_t^{(1)}, vp_t^{(2)}, \dots, vp_t^{(8)} \rangle$ of day t . Finally, according to the estimated volume profiles, it allocates a full-day executed parent order to eight 30-minute tranches.

B. Meta Trader

Within each tranche, slight fluctuations in market liquidity and the long execution time still degrade the performance of existing RL models. However, further splitting the tranche is susceptible to extreme short-term liquidity, enlarging the bias of volume profile estimation. Therefore, we establish Meta Trader to bridge the Macro Trader and the Micro Trader. On one hand, it fulfils each tranche's trading task given by the Macro Trader. On the other hand, it adjusts the order execution progress of the Micro Trader by issuing a subgoal appropriate for the short-term liquidity, where the subgoal works as a high-level action abstraction to simplify the Micro Trader's decision-making.

As shown in Fig. 4, the Meta Trader first selects the subgoal g^* with the highest expected reward from a preset subgoal set $\mathcal{G}$ based on current extrinsic state $\tilde{s}$ and its policy π_g to generate a mini-tranche. Assuming π_g is a greedy policy, the optimal

subgoal is selected by,

$$g^* = (T_{g^*}, V_{g^*}) = \arg \max_{g \in \mathcal{G}} Q(\tilde{s}, g; \theta), \quad (7)$$

where $Q(\cdot)$ is the state-action value function of Meta Trader estimated by a multilayer perceptron (MLP), T is the maximum step and V is the size of the order to be executed.

Then the Micro Trader receives g^* and executes child orders to fulfil g^* . At the end of each subgoal execution, the unfilled orders will be issued as a market order. The VWAP slippage during this period is used as $\tilde{r}$ that feeds back to the Meta Trader. We leverage DDQN [24] to iteratively update $Q(\cdot)$ and π_g . The DDQN consists of two independent state-action value functions Q_1 and Q'_1 with the same structure, where Q_1 is estimated by,

$$Q_1(\tilde{s}, g; \theta_1) = \max_{\pi_g} \mathbb{E} \left[\sum_{t'=t}^{t+N} \tilde{r}_{t'} + \gamma Q'_1(\tilde{s}_{t+N}, g'; \theta'_1) \mid \tilde{s}_t = \tilde{s}, g_t = g, \pi_g \right], \quad (8)$$

where $\tilde{r}_{t'}$ is the extrinsic reward of time step t' , and N is actual execution step of g , and g' is selected by (7). We use mean square error (MSE) as loss function to optimize Q_1 's and Q'_1 's parameters. Taking Q_1 as an example, its parameters θ_1 are updated by,

$$\theta_{1|t+N} = \arg \min_{\theta_{1|t}} \frac{1}{|D_1|} \sum_{(\tilde{s}, g, \tilde{r}, \tilde{s}') \sim D_1} \left(\sum_{t'=t}^{t+N} \tilde{r}_{t'} + \gamma Q'_1(\tilde{s}', g'; \theta'_{1|t}) - Q_1(\tilde{s}_t, g_t; \theta_{1|t}) \right)^2, \quad (9)$$

where D_1 is the experience replay of the Meta Trader consisting of $(\tilde{s}, g, \tilde{r}, \tilde{s}')$, and $|D_1|$ is the batch size sampled from D_1 .

C. Micro Trader

As the lowest layer of the M3T, the Micro Trader is used to execute each trading step. It optimizes order execution and finally fulfils the subgoal given by the Meta Trader. As shown in Fig. 5, the Micro Trader includes an multi-headed self-attention (MHSA) encoder (*a.k.a.* Transformer encoder [49]), an FC layer, and a subgoal encoder to process different sources of data. The MHSA encoder extracts the temporal representation of markets from the last 20 LOB snapshots. The FC layer maps the execution ratio into hidden representation. The subgoal encoder first uses one-hot encoding to encode the subgoal given by the Meta Trader and then learns a representation of the subgoal by an MLP. After that, another FC layer connects the above three modules and maps their outputs to state-action values.

The Micro Trader extends the general RL policy to include the subgoal given by the Meta Trader. At time t , it receives subgoal g given by the Meta Trader and decides order execution according to its policy π_{ag} . For instance, assuming π_{ag} is a greedy strategy, the order is executed by,

$$a^* = \arg \max_{a \in \mathcal{A}} Q(s, a; g, \theta). \quad (10)$$

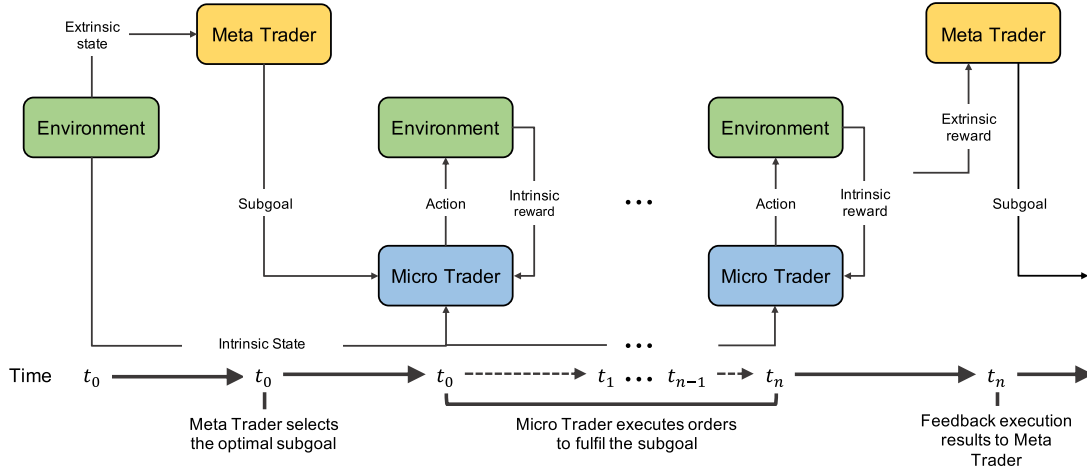


Fig. 4. Workflow of Meta Trader and Micro Trader. The Meta Trader first specifies an optimal subgoal for the Micro Trader. Then the Micro Trader executes orders in the simulation trading environment to fulfil the subgoal. After that, the Meta Trader gets an extrinsic reward as the subgoal's execution feedback and specifies a new optimal subgoal for the Micro Trader.

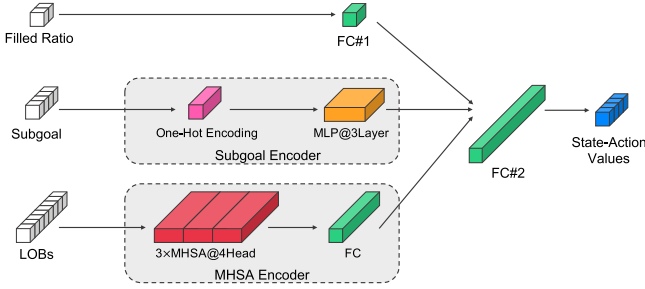


Fig. 5. Structure of Micro Trader, where the MHSA encoder includes three layers four-headed self-attention encoder, and MLP consists of three FC layers.

Similarly, the Micro Trader uses the DDQN to optimize its order execution policy and fulfil subgoals. The two independent state-action value functions Q_2 and Q'_2 of the Micro Trader are extended to include subgoals, where Q_2 is estimated by,

$$Q_2(s, a; g, \theta_2) = \max_{\pi_{ag}} \mathbb{E} \left[r_t + \gamma Q'_2(s', a'; g, \theta'_2) \mid s_t = s, a_t = a, g_t = g, \pi_{ag} \right], \quad (11)$$

where r_t is the intrinsic reward of time step t , and a' is selected by (10). We also use MSE as the loss function to optimize Q_2 's and Q'_2 's parameters. Taking Q_2 as an example, its parameters θ_2 are updated by,

$$\theta_{2|t+1} = \arg \min_{\theta_{2|t}} \frac{1}{|D_2|} \sum_{(s, a, r, s', g) \sim D_2} \left(r_t + \gamma \max_d Q'_2(s', d; g, \theta_{2|t}') - Q_2(s_t, a_t; g, \theta_{2|t}) \right)^2, \quad (12)$$

Algorithm 1. Parent Order Execution via M3T

Require: historical volume profile $\{vp_\tau^{(i)}\}_{\tau=t-20}^{t-1}$; parent order $\mathcal{T}$, Macro Trader $\mathcal{M}_{Macro}$, Meta Trader $\mathcal{M}_{Meta}$; Micro Trader $\mathcal{M}_{Micro}$; trading environment $\mathcal{E}_t$.

- 1: $\mathcal{M}_{Macro}$ estimates $vp_t^{(i)}$ based on $\{vp_\tau^{(i)}\}_{\tau=t-20}^{t-1}$;
- 2: $\mathcal{M}_{Macro}$ allocates $\mathcal{T}$ into tranches; $\{\mathcal{T}_t^{(i)}\}_{i=1}^8$
- 3: **for** $\mathcal{T}_t^{(i)}$ in $\{\mathcal{T}_t^{(i)}\}_{i=1}^8$ **do**
- 4: **while** $\mathcal{T}_t^{(i)}$ is unfilled **do**
- 5: $\mathcal{M}_{Meta}$ selects a subgoal g according to $\tilde{s}$;
- 6: **while** g is unfilled **do**
- 7: $\mathcal{M}_{Micro}$ executes orders in $\mathcal{E}_t$;
- 8: **end while**
- 9: **end while**
- 10: **end for**

where D_2 is the experience replay of the Micro Trader consisting of (s, a, g, r, s') , and $|D_2|$ is the batch size sampled from D_2 .

Algorithm 1 summarizes the process of executing a parent order $\mathcal{T}$ via the M3T, which reduces transaction costs by executing orders hierarchically.

V. EXPERIMENTS

Our experiments evaluate the transaction costs of different algorithmic trading strategies, volume profile estimation methods, as well as DL and RL backbones by back testing.

A. Dataset

From the SSE 50 Index's constituent stocks¹ in May 2014, we select ten representative stocks from different sectors and list them in Table II. These stocks show different price and daily volume ranges, indicating diverse trading patterns. We perform

¹<http://english.sse.com.cn/markets/dataservice/products/>

TABLE II
ID AND NAME OF THE STOCKS IN OUR EXPERIMENTS

Stock ID	Stock name	Sector	Minimum price	Maximum price	Daily average volume
600000	Shanghai Pudong Development Bank	Bank	11.71	6.22	338,817,801.86
600010	Inner Mongolia Baotou Steel Union	Materials	4.76	1.21	561,265,554.41
600018	Shanghai International Port	Industrials	8.09	4.37	69,325,028.28
600028	China Petroleum & Chemical Corporation	Energy	7.41	4.89	276,547,257.70
600030	Citic Securities Company Limited	Securities	37.25	11.33	330,351,951.36
600048	Poly Developments and Holdings Group	Property	11.90	4.83	200,242,099.37
600050	China United Network Communications	Telecom	5.87	3.11	243,544,130.42
600104	Saic Motor	Motor	25.98	14.28	35,998,667.48
600332	Guangzhou Baiyunshan Pharmaceutical	Healthcare	35.36	22.83	15,053,074.37
600519	Kweichow Moutai	Consumer	185.67	123.41	4,554,045.52

Shanghai Pudong Development Bank (600000)			
2014/6/3 9:30:01.000			
Level		Price	Size
Seller	Ask5	9.63	145540
	Ask4	9.62	157602
	Ask3	9.61	17500
	Ask2	9.60	357150
	Ask1	9.59	101643
Buyer	Bid1	9.58	441700
	Bid2	9.57	354100
	Bid3	9.56	223800
	Bid4	9.55	370300
	Bid5	9.54	251500

Fig. 6. Example of 5-level quote data of Shanghai Pudong Development Bank. It consists of five best selling price and five best buying price, as well as corresponding order sizes.

Shanghai Pudong Development Bank (600000)		
Time	Price	Size
9:30:00.011	9.59	800
9:30:00.100	9.59	800
9:30:00.130	9.6	200
9:30:00.130	9.6	800
9:30:00.240	9.59	1575
9:30:00.240	9.59	10000
9:30:00.240	9.59	500
9:30:00.240	9.59	300
9:30:00.240	9.59	200
9:30:00.240	9.59	19700
9:30:00.240	9.59	600
9:30:00.240	9.59	50000
9:30:00.370	9.59	800
9:30:00.970	9.59	500

Fig. 7. Example of trade data of Shanghai Pudong Development Bank. It records the price and size of the orders that have been filled.

experiments on their level-2 data (consisting of LOBs and trade) for each trading day from June 2014 to March 2015. The LOBs are snapshots of quotes, with about 7.5 million overall, while the number of trading data is about 625 million. Figs. 6 and 7 illustrate the structure of a quote sample and trade records between two snapshots. We exclude the data of the days when the market price reaches the price limit due to lack of liquidity.

B. Baselines and Metrics

To verify the effectiveness of the Macro Trader in volume profile estimation, we compare it with a baseline volume profile estimation method and two DL models:

TABLE III
STRUCTURE SETTING OF BASELINES AND M3T

model	Macro Trader (Profile Estimation)	Meta Trader (Subgoal Selection)	Micro Trader (Order Execution)
VWAP	MA	N/A	Rule-based
AP	MA	N/A	Rule-based
DQN	LSTM	N/A	DQN@MHSA
DDQN	LSTM	N/A	DDQN@MHSA
A2C	LSTM	N/A	A2C@MHSA
M3T	LSTM	DDQN@MLP	DDQN@MHSA

- **MA**: Use the average value of the last N days (20 days in our experiments) as the estimate of the future. It is a common method to predict the volume profile in the foundational VWAP strategy.
- **FC**: A neural network consisting of a single layer of learnable parameters, *a.k.a.* linear layer.
- **MLP**: A neural network consisting of several layers of learnable parameters.

We compare our M3T with two classical algorithmic trading strategies and three deep RL models:

- **VWAP**: The order is allocated into tranches according to volume profiles to track the VWAP.
- **Arrival Price (AP)**: Order's price is determined by the filled ratio, i.e., if the filled ratio is lower than the time ratio, execute as a market order; otherwise, execute as a limit order. When the filled ratio exceeds 10% of the time ratio, it stops issuing new orders.
- **DQN** [20]: A TD-based deep RL model. [11], [26], [31] used it to optimize algorithmic trading strategies.
- **DDQN** [24]: An improved deep RL model based on DQN. Ning et al. [12] used it to optimize algorithmic trading strategies.
- **Advantage Actor-Critic (A2C)** [50]: An actor-critic-based deep RL model, which consists of an actor network and a critic network.

The classical algorithmic trading strategies use historical volume profiles' MA to estimate future volume profiles, while the DRL models use LSTM. Table III summarizes the settings of these baselines and the M3T in our experiments.

We use the VWAP slippage presented in (2) to measure the transaction costs and propose *subgoal speed* to evaluate the overall liquidity required by subgoals within each tranche, that is,

$$u = \frac{\sum_{g \in \text{tranche}_i} ((V_g/T_g) \cdot n_g)}{\sum_{g \in \text{tranche}_i} n_g}, \quad (13)$$

TABLE IV
HYPERPARAMETERS OF M3T AND BASELINES

Model	Hyperparameter	Best
Macro Trader	Epoch	5,000
	Batch size	128
	Learning rate	1e-4
RL Training	Episode	10,000
	Discount	0.99
	Learning rate	5e-5
	Order size within each tranche	100K ~ 200K
Meta Trader	Initial ϵ	1.00
Micro Trader	ϵ decline rate	0.99/5 Episodes
DQN	Experience replay capacity	1e4
DDQN	Experience replay batch size	128
A2C	Synchronized environments	16

where n_g is the times that Meta Trader selects subgoal g within tranche $\#i$.

C. Training Setup

In Macro Trader training, we use the first 60% of the dataset (June 2014 to October 2014) as the training set, the middle 20% of the dataset (November 2014 to December 2014) as the validation set, and the last 20% of the dataset (January 2015 to March 2015) as the test set. In Meta Trader and Micro Trader training, we use the first 80% of the dataset (June 2014 to December 2014) as the training set and the last 20% of the dataset (January 2015 to March 2015) as the test set.

We train the M3T, DQN, and DDQN with ϵ -greedy and experience replay to improve these models' performance. The M3T and the baselines are trained by the same hyperparameters selected according to the performance of DDQN on *Shanghai Pudong Development Bank (600000)*, as shown in Table IV. Considering the stocks' limit order books and liquidity, we set the empirical waiting queue length ι as 3.

D. Experimental Results

1) *Volume Profile Estimation*: In the VWAP strategy, parent orders are first allocated into several tranches according to the volume profiles to reduce the liquidity risk. It is important to estimate the volume profiles accurately. To reveal the performance of macro traders in volume profile estimation, we perform experiments on the test set and calculate the MSE of the estimated volume profiles. Table V shows the MSE of the volume profile estimation over ten stocks, where the unit of values is millesimal (i.e., 1e-3). Each stock's best and second-best results are highlighted in bold and underlined. The experimental results reveal that the Macro Trader has the lowest MSE over all stocks, confirming its effectiveness in volume profile estimation. The baseline method MA shows the second-best performance and far outperforms FC and MLP.

2) *Transaction Costs*: We evaluate the transaction costs of M3T and baselines by simulating trading 120,000 shares each day. To closely reveal the performance of trading strategies in real-world stock markets, we evaluate their performance in a simulated online scenario. The evaluation data are organized in chronological order. Each model is evaluated on these data one

by one. There is no leakage of future data, ensuring that the strategies' action is close to that in the real-world markets.

Table VI compares transaction costs of the M3T and the baselines on ten stocks in terms of VWAP slippage (bp). The results show that the M3T outperforms all the baselines and achieves the lowest transaction costs over seven stocks. DDQN is the best baseline, achieving second-best over four stocks, while DQN achieves second-best over two stocks. The AP's general performance is close to the DQN and slightly better than the A2C. The foundational VWAP strategy has the highest transaction costs. These results demonstrate that using RL models to optimize the trading strategies can reduce transaction costs. Furthermore, even though the DDQN adapts the same RL backbone model as the M3T, the M3T's transaction costs are lower than the DDQN by 2.22 bp at most. This result demonstrates the effectiveness of the Meta Trader. Splitting each tranche into several subgoals can effectively reduce the transaction costs of long-duration orders.

3) *Computing Costs*: Since market opportunities are fleeting, algorithmic trading's time efficiency is a critical indicator in practice. To reveal the time efficiency of M3T, we measure the time costs of training M3T and baselines on an Nvidia Titan XP GPU. Table X shows the average time cost of training Macro Trader is less than 10 seconds per 1000 epochs, which is acceptable for users because the Macro Trader only needs to train offline once before executing orders. Table XI shows that even though the M3T includes an additional hierarchical trader (i.e., the Meta Trader) compared to the DDQN, the average cost for training M3T is similar to the DQN and the DDQN used in previous studies, and it is lower than the A2C.

E. Analysis

Although we have demonstrated the effectiveness of M3T in reducing transaction costs via previous experiments, we would like to figure out the effectiveness of each trader in the M3T. In other words, we are concerned with the following three questions:

- 1) Can precisely predicting volume profiles reduce transaction costs?
- 2) How do different RL backbones affect M3T's performance?
- 3) How do different DL backbones affect M3T's performance?

To answer the above three questions empirically, we added the following three analytical experiments:

Volume Profile Estimation: To explore the impact of different volume profile estimation methods on the transaction costs, we estimate the volume profiles by MA and Macro Trader (i.e., LSTM) respectively and compare their final transaction costs over the M3T and the baselines in Table VII. The value highlighted in bold indicates that the final transaction costs of using LSTM are lower than using MA, while the value highlighted underlined is the opposite. The experimental results demonstrate that the transaction costs will increase without the Macro Traders, highlighting its effectiveness. Referring to the

TABLE V
MSE (1E-3) OF MACRO TRADER AND BASELINES IN VOLUME PROFILE ESTIMATION

Model	600000	600010	600018	600028	600030	600048	600050	600104	600332	600519
MA	2.48	3.41	4.43	2.18	3.05	2.32	2.71	1.64	2.45	1.36
Linear	3.09	3.40	4.59	2.61	4.41	2.63	3.49	1.79	3.33	1.69
MLP	4.66	6.02	5.86	4.22	6.85	5.31	4.62	2.92	4.25	4.36
Macro Trader	2.27	3.17	4.20	2.16	2.96	2.06	2.68	1.62	2.30	1.35

The value highlighted in bold represents the best performance among all methods.

TABLE VI
VWAP SLIPPAGE (BP) OF M3T AND BASELINES IN PARENT ORDER EXECUTION

Model	600000	600010	600018	600028	600030	600048	600050	600104	600332	600519
VWAP	3.37±4.25	10.47±8.77	9.01±6.59	8.21±6.05	2.97±6.31	6.64±5.88	11.38±12.12	3.23±5.80	4.40±6.98	-0.17±3.33
AP	-2.41±4.65	-5.37±7.97	1.58±6.33	-5.77±7.02	-0.51±4.18	-1.57±6.06	-2.41±10.89	-0.10±5.34	2.78±9.95	-0.28±3.99
DQN	-3.16±3.73	-10.64±6.59	-6.05±3.58	-3.01±5.12	-2.33±5.49	-1.92±6.73	4.89±9.85	-3.00±6.21	3.85±10.05	-0.31±2.64
DDQN	-3.53±3.99	-10.92±6.22	-7.60±5.98	-4.51±6.78	-2.32±4.97	-2.31±4.63	-0.33±6.63	-2.98±7.00	2.52±8.32	-0.32±1.97
A2C	-2.35±3.95	-4.75±8.03	-3.58±9.28	-3.23±5.88	-1.97±4.22	0.85±5.73	2.42±8.03	-2.38±4.54	3.31±9.88	-0.27±3.45
M3T	-4.38±3.76	-12.18±7.00	-9.42±6.12	-5.23±4.89	-3.01±5.28	-3.32±6.11	-2.55±8.76	-3.66±5.45	2.05±7.66	-0.41±2.38

The value highlighted in bold represents the best performance among all methods.

TABLE VII
VWAP SLIPPAGE (BP) OF DIFFERENT VOLUME PROFILE ESTIMATION MODELS

Model	600000	600010	600018	600028	600030	600048	600050	600104	600332	600519
VWAP@MA	3.37	10.47	9.01	8.21	2.97	6.64	11.38	3.23	4.40	-0.17
VWAP@LSTM	3.35	10.51	8.92	8.21	2.97	6.63	11.37	3.20	4.10	-0.19
$\Delta_{(LSTM-MA)}$	-0.02	+0.04	-0.09	0.00	0.00	-0.01	-0.01	-0.03	-0.30	-0.02
AP@MA	-2.41	-5.37	1.58	-5.77	-0.51	-1.57	-2.41	-0.10	2.78	-0.28
AP@LSTM	-2.42	-5.38	1.62	-5.80	-0.50	-1.60	-2.38	-0.06	2.62	-0.25
$\Delta_{(LSTM-MA)}$	-0.01	+0.01	+0.04	-0.03	+0.01	-0.03	+0.03	+0.04	-0.16	+0.03
DQN@MA	-3.12	-10.17	-6.02	-2.89	-2.12	-1.77	5.28	-2.56	3.88	-0.30
DQN@LSTM	-3.16	-10.64	-6.05	-3.01	-2.33	-1.92	4.89	-3.00	3.85	-0.31
$\Delta_{(LSTM-MA)}$	-0.04	-0.47	-0.03	-0.12	-0.21	-0.15	-0.39	-0.44	-0.03	-0.01
DDQN@MA	-3.47	-10.83	-7.44	-4.50	-2.33	-2.19	-0.29	-2.73	2.57	-0.30
DDQN@LSTM	-3.53	-10.92	-7.60	-4.51	-2.32	-2.31	-0.33	-2.98	2.52	-0.32
$\Delta_{(LSTM-MA)}$	-0.06	-0.09	-0.16	-0.01	+0.01	-0.12	-0.04	-0.25	-0.05	-0.02
A2C@MA	-2.33	-4.70	-3.47	-3.21	-1.99	0.92	2.47	-2.39	3.33	-0.28
A2C@LSTM	-2.35	-4.75	-3.58	-3.23	-1.97	0.85	2.42	-2.38	3.31	-0.27
$\Delta_{(LSTM-MA)}$	-0.02	-0.05	-0.11	-0.02	+0.02	-0.07	-0.05	+0.01	-0.02	-0.01
M3T@MA	-4.35	-11.09	-9.39	-5.18	-3.03	-3.09	-2.32	-3.51	2.06	-0.41
M3T@LSTM	-4.38	-12.18	-9.42	-5.23	-3.01	-3.32	-2.55	-3.66	2.05	-0.41
$\Delta_{(LSTM-MA)}$	-0.03	-0.09	-0.03	-0.05	+0.02	-0.13	-0.23	-0.15	-0.01	0.00

The value highlighted in bold represents the best performance among all methods.

TABLE VIII
VWAP SLIPPAGE (BP) OF M3T BASED ON DIFFERENT DL MODELS

Model	600000	600010	600018	600028	600030	600048	600050	600104	600332	600519
M3T@FC	-3.21	85.12	-6.29	8.72	-0.88	-0.73	38.24	-0.03	2.92	-0.30
M3T@CNN	3.58	13.45	2.15	7.23	0.50	2.12	28.45	1.75	3.43	-0.07
M3T@LSTM	-4.12	-11.22	-8.63	-4.25	-2.33	-3.47	-1.53	-3.47	2.96	-0.40
M3T@MHSA	-4.38	-12.18	-9.42	-5.23	-3.01	-3.32	-2.55	-3.66	2.05	-0.41

The value highlighted in bold represents the best performance among all methods.

results of Table V, we can conclude that accurately estimating volume profiles effectively reduces transaction costs.

Deep Learning Backbone: To reveal the impact of different DL models on the M3T's performance, we establish Micro Traders based on several DL models such as FC, CNN, and LSTM as comparisons. The results in Table VIII demonstrate that the DL backbones can affect the performance of M3T with almost consistent differences across all stocks. The MHSA performs best, the LSTM is second, and the CNN performs the worst.

Reinforcement Learning Backbone: To reveal the impact of different RL models on the M3T's performance, we train a Meta Trader and a Micro Trader based on DQN as comparisons. The

results shown in Table IX demonstrate that DDQN-based M3T outperforms DQN on almost all stocks.

F. Case Study

To analyze whether the Meta Trader can select appropriate subgoals according to the market's liquidity, we count the subgoals selected by the Meta Trader of the best-performed M3T (i.e., the M3T with the lowest transaction cost over checkpoints) in each tranche and calculate their *subgoal speed*, as shown in Fig. 8. The curve in Fig. 8 shows that the *subgoal speed* of the M3T@DQN and the M3T@DDQN is generally between 0.16 and 0.17. It shows the same pattern on most stocks, with higher *subgoal speed* in the middle and lower on both sides,

TABLE IX
VWAP SLIPPAGE (BP) OF M3T BASED ON DIFFERENT RL MODELS

Model	600000	600010	600018	600028	600030	600048	600050	600104	600332	600519
M3T@DQN	-3.45	-10.95	-8.83	-5.51	-2.97	-3.21	-2.25	-3.39	2.47	-0.36
M3T@DDQN	-4.38	-12.18	-9.42	-5.23	-3.01	-3.32	-2.55	-3.66	2.05	-0.41

The value highlighted in bold represents the best performance among all methods.

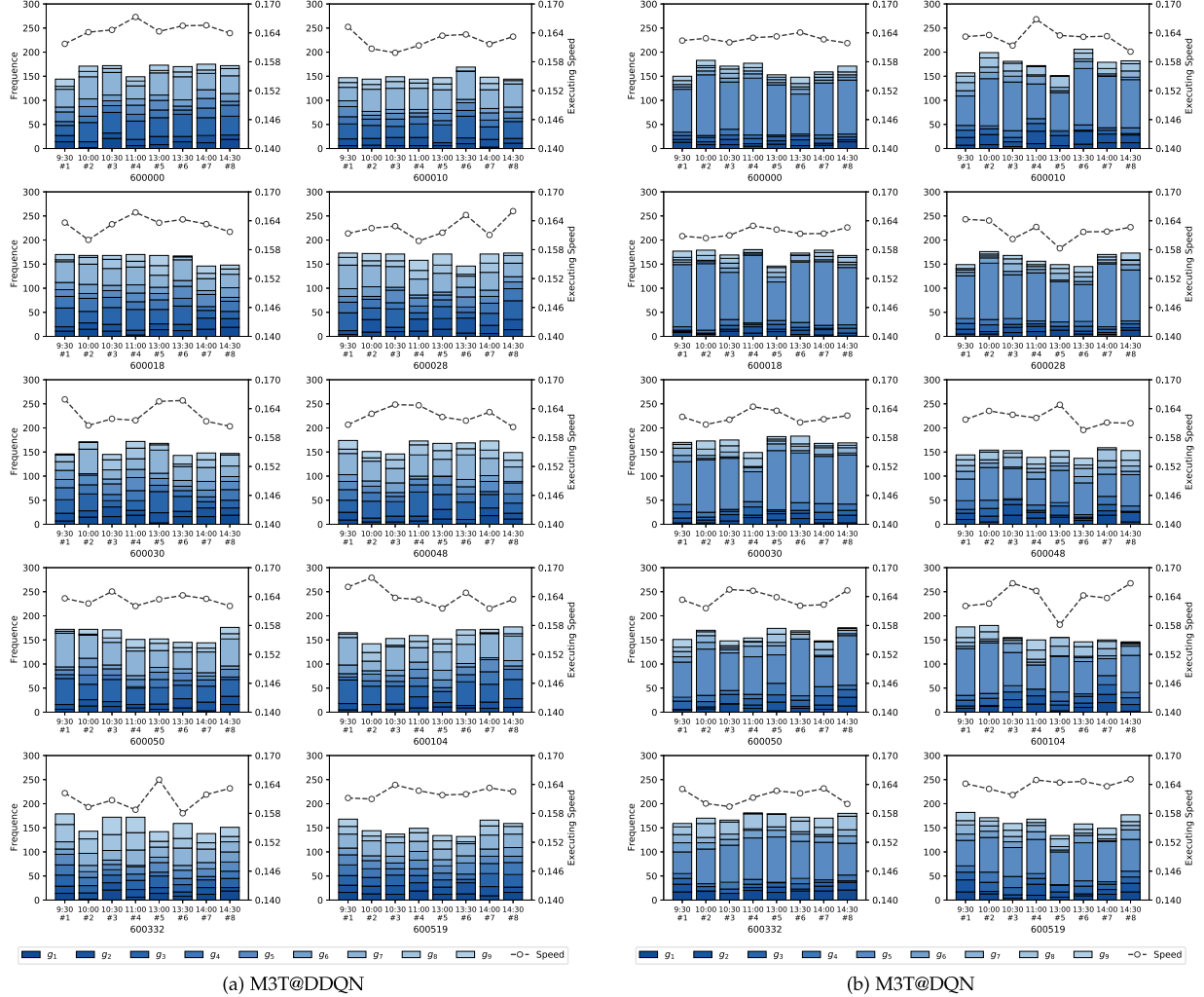


Fig. 8. Subgoal counts and subgoal speed over ten stocks.

TABLE X
TIME COSTS (PER 1000 EPOCHS) OF TRAINING MACRO TRADER

Macro Trader	Linear	MLP	LSTM
Time costs	1.23s±0.22	10.65s±0.95	7.44s±0.77

TABLE XI
TIME COSTS (PER EPISODE) OF TRAINING M3T AND BASELINES

models	DQN	DDQN	A2C	M3T
Time costs	81.75s±19.88	87.08s±16.34	126.59s±18.21	98.42s±21.06

indicating that the M3T@DQN and the M3T@DDQN tend to fulfil orders quickly in the less liquid period. By comparing the number of subgoals selected by the M3T@DQN and the M3T@DDQN, we find that the M3T@DQN tends to select

the subgoals with balanced *subgoal speed*, such as subgoal #5, while the M3T@DDQN tends to select the subgoals with relatively slower (e.g., subgoal #3) or faster *subgoal speed* (e.g., subgoal #7). Combining with the results shown in Table VI, it demonstrates that the Meta Trader can reduce the transaction costs by adjusting order execution speed to keep consistency with the short-term liquidity.

VI. CONCLUSION

In this paper, we address a crucial concern of brokers in designing an intelligent VWAP strategy to minimize transaction costs of long-duration orders. While previous studies have shown that using RL for order execution can reduce transaction costs, these findings are limited to orders with short duration and small sizes. The intraday liquidity pattern changes and

sparse reward signals limit their performance in the execution of long-duration orders. To overcome the limitations of existing RL models, we propose M3T, a joint model inspired by the temporal abstraction concept of HRL. The M3T consists of three traders that capture liquidity patterns and execute orders at different temporal scales, enhancing the VWAP strategy through parent order allocation (Macro Trader), subgoal selection (Meta Trader), and child order execution (Micro Trader). Experimental results demonstrated that 1) the M3T outperforms the baselines, with up to 2.22 base points cost saving compared to the optimal baseline DDQN; 2) the Macro Trader can improve the accuracy of volume profile estimation and reduce the transaction costs; 3) each trader in the M3T is effective to reduce the transaction costs. The additional subgoal selection analysis demonstrates that the Meta Trader can reduce transaction costs by selecting appropriate subgoals according to market liquidity.

It is worth noting that the performance of our model may be affected by inference speed, like other DRL-based algorithmic trading strategies. The expensive computational overhead can potentially lead to delays in real-time decision-making. We recommend that users assess the minimal data interval required to ensure that the time taken for inference remains within acceptable limits. If the inference time exceeds this interval, there is a risk that the agent's actions may not align accurately with the rapidly changing market conditions. Besides, as many studies point out, pattern shifts are prevalent in stock markets [51], [52], [53]. These shifts can challenge the effectiveness of models trained on historical data. A common industry practice is re-training models with the new data after each day's trading, while the efficient alternative approach needs further investigation. We believe additional multi-modal information sources, such as news sentiments [54], could improve the performance and robustness of the trading strategy to pattern shifts. In addition, we assume the agent orders have a negligible impact on the market, like most related studies. However, it may deviate greatly from the actual situation under extreme liquidity conditions.

In future work, we plan to improve the RL agent's performance by quantifying the impact of news events on the market as an additional market state and incorporating the neural model search to optimize M3T's hyperparameters. The effect of different HRL variants on the performance of M3T is also worthy of investigation. Besides, it is still an open question to study how to more precisely estimate simulated orders' impact on historical stock markets.

REFERENCES

- [1] S. A. Berkowitz, D. E. Logue, and E. A. Noser Jr, "The total cost of transactions on the NYSE," *J. Finance*, vol. 43, no. 1, pp. 97–112, 1988.
- [2] B. Johnson, *Algorithmic Trading & DMA: An Introduction to Direct Access Trading Strategies*. 4Myeloma Press, 2010.
- [3] O. Guéant and G. Royer, "VWAP execution and guaranteed VWAP," *SIAM J. Financial Math.*, vol. 5, no. 1, pp. 445–471, 2014.
- [4] S. Lin and P. A. Beling, "An end-to-end optimal trade execution framework based on proximal policy optimization," in *Proc. 29th Int. Joint Conf. Artif. Intell.*, 2020, pp. 4548–4554.
- [5] K. Dabérius, E. Granat, and P. Karlsson, "Deep execution-value and policy based reinforcement learning for trading and beating market benchmarks," *Available SSRN 3374766*, pp. 1–32, 2019.
- [6] H. Konishi, "Optimal slice of a VWAP trade," *J. Financial Markets*, vol. 5, no. 2, pp. 197–221, 2002.
- [7] J. Bialkowski, S. Darolles, and G. Le Fol, "Improving VWAP strategies: A dynamic volume approach," *J. Bank. Finance*, vol. 32, no. 9, pp. 1709–1722, 2008.
- [8] S. M. Kakade, M. Kearns, Y. Mansour, and L. E. Ortiz, "Competitive algorithms for VWAP and limit order trading," in *Proc. 5th ACM Conf. Electron. Commerce*, 2004, pp. 189–198.
- [9] R. Almgren and J. Lorenz, "Bayesian adaptive trading with a daily cycle," *J. Trading*, vol. 1, no. 4, pp. 38–46, 2006.
- [10] Á. Cartea and S. Jaimungal, "A closed-form execution strategy to target volume weighted average price," *SIAM J. Financial Math.*, vol. 7, no. 1, pp. 760–785, 2016.
- [11] Y. Nevmyvaka, Y. Feng, and M. Kearns, "Reinforcement learning for optimized trade execution," in *Proc. 23rd Int. Conf. Mach. Learn.*, 2006, pp. 673–680.
- [12] B. Ning, F. H. T. Lin, and S. Jaimungal, "Double deep Q-learning for optimal execution," *Appl. Math. Finance*, vol. 28, no. 4, pp. 361–380, 2021.
- [13] T. A. Ferreira, "Reinforced deep Markov models with applications in automatic trading," pp. 1–39, 2020, *arXiv: 2011.04391*.
- [14] T. Jaisson, "Deep differentiable reinforcement learning and optimal trading," *Quantitative Finance*, vol. 22, no. 8, pp. 1429–1443, 2022.
- [15] S. Jaimungal, S. M. Pesenti, Y. S. Wang, and H. Tatsat, "Robust risk-aware reinforcement learning," *SIAM J. Financial Math.*, vol. 13, no. 1, pp. 213–226, 2022.
- [16] A. Coache, S. Jaimungal, and A. Cartea, "Conditionally elicitable dynamic risk measures for deep reinforcement learning," *SIAM J. Financial Math.*, vol. 14, no. 4, pp. 1249–1289, 2023.
- [17] A. Coache and S. Jaimungal, "Reinforcement learning with dynamic convex risk measures," *Math. Finance*, vol. 10, no. 1, pp. 1–31, 2023.
- [18] V. Niederhoffer and M. F. M. Osborne, "Market making and reversal on the stock exchange," *J. Amer. Stat. Assoc.*, vol. 61, no. 316, pp. 897–916, 1966.
- [19] J. Hasbrouck, "The dynamics of discrete bid and ask quotes," *J. Finance*, vol. 54, no. 6, pp. 2109–2142, 1999.
- [20] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [21] T. D. Kulkarni, K. Narasimhan, A. Saeedi, and J. Tenenbaum, "Hierarchical deep reinforcement learning: Integrating temporal abstraction and intrinsic motivation," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2016, pp. 3675–3683.
- [22] Y. Ma et al., "A hierarchical reinforcement learning based optimization framework for large-scale dynamic pickup and delivery problems," *Adv. Neural Inf. Process. Syst.*, vol. 34, pp. 23609–23620, 2021.
- [23] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [24] H. Van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. AAAI Conf. Artif. Intell.*, 2016, pp. 2094–2100.
- [25] D. Bertsimas and A. W. Lo, "Optimal control of execution costs," *J. Financial Markets*, vol. 1, no. 1, pp. 1–50, 1998.
- [26] R. Almgren and N. Chriss, "Optimal execution of portfolio transactions," *J. Risk*, vol. 3, pp. 5–40, 2001.
- [27] G. Huberman and W. Stanzl, "Optimal liquidity trading," *Rev. Finance*, vol. 9, no. 2, pp. 165–200, 2005.
- [28] D. Hendricks and D. Wilcox, "A reinforcement learning extension to the almgren-chriss framework for optimal trade execution," in *Proc. IEEE Conf. Comput. Intell. Financial Eng. Econ.*, 2014, pp. 457–464.
- [29] Z. Ye, W. Deng, S. Zhou, Y. Xu, and J. Guan, "Optimal trade execution based on deep deterministic policy gradient," in *Proc. Int. Conf. Database Syst. Adv. Appl.*, Springer, 2020, pp. 638–654.
- [30] J. Ye, X. Li, and Y. Wang, "A multi-agent deep reinforcement learning framework for VWAP strategy optimization," in *Proc. Int. Joint Conf. Neural Netw.*, 2022, pp. 1–8.
- [31] Y. Shen, R. Huang, C. Yan, and K. Obermayer, "Risk-averse reinforcement learning for algorithmic trading," in *Proc. IEEE Conf. Comput. Intell. Financial Eng. Econ.*, 2014, pp. 391–398.
- [32] Y. Fang et al., "Universal trading for order execution with oracle policy distillation," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 107–115.
- [33] R. Wang, H. Wei, B. An, Z. Feng, and J. Yao, "Commission fee is not enough: A hierarchical reinforced framework for portfolio management," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 626–633.

- [34] P. Dayan and G. Hinton, "Feudal reinforcement learning," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, Morgan Kaufmann, 1993, pp. 271–278.
- [35] R. S. Sutton, D. Precup, and S. Singh, "Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning," *Artif. Intell.*, vol. 112, no. 1/2, pp. 181–211, 1999.
- [36] J. Sorg and S. Singh, "Linear options," in *Proc. 9th Int. Conf. Auton. Agents Multiagent Syst.*, International Foundation for Autonomous Agents and Multiagent Systems, 2010, pp. 31–38.
- [37] C. Szepesvari et al., "Universal option models," *Adv. Neural Inf. Process. Syst.*, vol. 27, pp. 990–998, 2014.
- [38] P.-L. Bacon, J. Harb, and D. Precup, "The option-critic architecture," in *Proc. AAAI Conf. Artif. Intell.*, vol. 2017, pp. 1726–1734.
- [39] S. Mohamed and D. J. Rezende, "Variational information maximisation for intrinsically motivated reinforcement learning," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2015, pp. 2125–2133.
- [40] B. Eysenbach, A. Gupta, J. Ibarz, and S. Levine, "Diversity is all you need: Learning skills without a reward function," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–22.
- [41] A. Goyal, S. Sodhani, J. Binas, X. B. Peng, S. Levine, and Y. Bengio, "Reinforcement learning with competitive ensembles of information-constrained primitives," in *Proc. Int. Conf. Learn. Representations*, 2019, pp. 1–21.
- [42] T. Schaul, D. Horgan, K. Gregor, and D. Silver, "Universal value function approximators," in *Proc. Int. Conf. Mach. Learn.*, 2015, pp. 1312–1320.
- [43] O. Nachum, S. Gu, H. Lee, and S. Levine, "Data-efficient hierarchical reinforcement learning," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2018, pp. 3307–3317.
- [44] M. Andrychowicz et al., "Hindsight experience replay," *Adv. Neural Inf. Process. Syst.*, vol. 30, pp. 1–11, 2017.
- [45] A. Levy, G. Konidaris, R. Platt, and K. Saenko, "Learning multi-level hierarchies with hindsight," in *Proc. Int. Conf. Learn. Representations*, 2019, pp. 1–16.
- [46] R. Donnelly and L. Gan, "Optimal decisions in a time priority queue," *Appl. Math. Finance*, vol. 25, no. 2, pp. 107–147, 2018.
- [47] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: MIT Press, 2018.
- [48] J. J. Hopfield, "Neural networks and physical systems with emergent collective computational abilities," *Proc. Nat. Acad. Sci.*, vol. 79, no. 8, pp. 2554–2558, 1982.
- [49] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 5998–6008.
- [50] Y. Wu, E. Mansimov, S. Liao, A. Radford, and J. Schulman, "Openai baselines: ACKTR & A2C," 2017. [Online]. Available: url: <https://openai.com/blog/baselines-acktr-a2c>
- [51] A. Botte and D. Bao, "A machine learning approach to regime modeling," *Two Sigma*, 2021. [Online]. Available: <https://www.twosigma.com/wp-content/uploads/2021/10/Machine-Learning-Approach-to-Regime-Modeling.pdf>
- [52] X. Li and P. Wu, "Stock price prediction incorporating market style clustering," *Cogn. Comput.*, vol. 14, no. 1, pp. 149–166, 2022.
- [53] P. Wu and X. Li, "Market style discrimination via ensemble learning," in *Proc. IEEE 13th Int. Conf. Softw. Eng. Serv. Sci.*, 2022, pp. 170–173.
- [54] X. Li, P. Wu, and W. Wang, "Incorporating stock prices and news sentiments for stock market prediction: A case of Hong Kong," *Inf. Process. Manage.*, vol. 57, no. 5, 2020, Art. no. 102212.



of China.

Xiaodong Li received the PhD degree in computer science from the City University of Hong Kong. He is an associate professor with the College of Computer and Information at Hohai University. His research interests include artificial intelligence, machine learning, information retrieval, and data mining, especially Big Data applications to algorithmic trading in finance and water resource management in hydrology. He has served as a PI of a sub-project with the National Key R&D Program of China, and a PI of a project with the National Natural Science Foundation



Pangjing Wu received the BS degree in information and computing science from Hohai University, and the ME degree from the College of Computer and Information. His research interests include reinforcement learning, financial engineering, and especially Big Data applications to algorithmic trading in finance. He has participated in the FinTech research group under the supervision of Dr. Xiaodong Li.



Chenxin Zou received the bachelor's degree in computer science and technology from Hohai University. He is currently working toward the postgraduate degree with the College of Computer and Information, Hohai University. His research interests include machine learning and knowledge graph, especially construction of domain knowledge graph. He has participated in the FinTech Research Group, under the supervision of Dr. Xiaodong Li.



Qing Li (Fellow, IEEE) received the MSc and PhD degrees in computer science from the University of Southern California, Los Angeles. He is currently a chair professor with the Department of Computing, Hong Kong Polytechnic University. His research interests include multi-modal data management, conceptual data modeling, social media, Web services, and e-learning systems. He has authored more than 400 publications. He is actively involved in the research community. He is a fellow of IEEE/IET, U.K., and a distinguished member of CCF, China. He served as a conference and program chair/co-chair for numerous major international conferences. He also sits in the Steering Committees of DASFAA, ER, ACM RecSys, IEEE U-MEDIA, and ICWL. He has served as an associate editor for a number of major technical journals, including *IEEE Transactions on Knowledge and Data Engineering*, *ACM Transactions on Internet Technology*, *Data Science and Engineering*, *World Wide Web*, and *Journal of Web Engineering*.